

Mini-Projet Intelligence Artificielle
Master 1 (M1) — Parcours Informatique

Smart Pot

Système Expert pour la Gestion Intelligente des Plantes d'Intérieur

Domaine : Intelligence Artificielle — Systèmes Experts
Moteur utilisé : Chaînage avant (Forward Chaining)
Langage : Python 3 / Tkinter
Base de faits : 5 espèces de plantes grasses / succulentes
Base de règles : 25 règles de production
Référence RHS : Royal Horticultural Society

Réalisé par :
Amen Allah Hadded

Table des matières

Liste des figures	3
Introduction générale	4
1 Contexte et objectifs du projet	5
1.1 Problématique	6
1.2 Objectifs du système expert	6
1.3 Choix du domaine d'application	7
1.4 Démarche de recherche documentaire	7
2 Base de connaissances	8
2.1 Base de faits	9
2.1.1 Plantes référencées dans la base	9
2.1.2 Structure détaillée d'un fait — Exemple de l'Aloe Vera	9
2.2 Base de règles de production	10
2.2.1 Règles d'identification (R01 à R05)	10
2.2.2 Règles de classification (R06 à R10)	10
2.2.3 Règles d'arrosage (R11 à R15)	11
2.2.4 Règles d'alerte (R16 à R22)	11
2.2.5 Règles de précaution (R23 à R25)	12
3 Moteur d'inférence	13
3.1 Principe du chaînage avant	14
3.1.1 Flux de raisonnement séquentiel	14
3.2 Implémentation des quatre étapes	15
3.2.1 Étape 1 — Identification de la plante	15
3.2.2 Étape 2 — Classification morphologique	15
3.2.3 Étape 3 — Calcul des fréquences d'arrosage	15
3.2.4 Étape 4 — Évaluation des alertes et précautions	16
4 Interface utilisateur et résultats	17
4.1 Architecture de l'interface graphique	18
4.1.1 Panneau de saisie — Entrées utilisateur	18
4.2 Déroulement du diagnostic	19
4.2.1 Affichage des résultats	19
Conclusion	20

Références documentaires

21

Table des figures

4.1	Interface graphique principale de l'application Smart Pot	18
4.2	Affichage des résultats du diagnostic	19

Introduction générale

Ce rapport présente la conception et la réalisation d'un système expert dédié à la gestion intelligente des plantes grasses et succulentes, baptisé « Smart Pot ». Face au problème récurrent de l'arrosage des plantes lors des absences prolongées (vacances, missions professionnelles), ce projet propose une solution logicielle capable de reconnaître automatiquement une plante à partir de ses caractéristiques morphologiques, de déterminer sa fréquence d'arrosage adaptée selon la saison, et de générer des alertes en cas de conditions défavorables (gel, sur-arrosage, infestation, toxicité). Le système repose sur une base de connaissances de 5 espèces documentées via les fiches de la Royal Horticultural Society (RHS) et 25 règles de production. Le moteur d'inférence implémente un chaînage avant organisé en quatre étapes séquentielles : identification, classification, calcul de l'arrosage et évaluation des alertes. L'interface utilisateur développée avec Tkinter permet une saisie intuitive des observations et une visualisation claire des résultats sous forme de fiche plante, planning mensuel et alertes classées par criticité.

1.

Contexte et objectifs du projet

Contexte et objectifs du projet

Ce premier chapitre présente la problématique à l'origine du projet Smart Pot, ainsi que les objectifs fixés par le cahier des charges. Il justifie également le choix du domaine d'application et introduit les sources documentaires utilisées pour construire la base de connaissances.

1.1 Problématique

De nombreuses personnes se retrouvent régulièrement confrontées à une question pratique lors de départs en vacances ou en mission professionnelle : *qui va s'occuper des plantes en leur absence ?* En effet, trouver une personne de confiance capable d'arroser les plantes selon leurs besoins spécifiques est souvent difficile, coûteux et peu fiable. Cette difficulté est d'autant plus grande que les besoins en eau varient considérablement d'une espèce à l'autre.

L'idée du **Smart Pot** répond directement à cette problématique en proposant un pot intelligent autonome, capable de reconnaître la plante qu'il contient, de déterminer sa fréquence d'arrosage et de générer des alertes en cas d'anomalie. Dans le cadre de ce mini-projet de Master 1, nous nous concentrons sur la **partie logicielle** : le système expert qui constitue le cerveau décisionnel du dispositif. Cette approche permet de séparer clairement la phase de conception algorithmique de la future intégration matérielle.

1.2 Objectifs du système expert

Le système expert développé doit remplir les quatre fonctions principales définies dans le cahier des charges du projet Smart Pot :

1. **Reconnaissance de la plante** — à partir de caractéristiques observables simples (forme générale, type de feuilles, présence d'épines, hauteur estimée), le système identifie l'espèce par un mécanisme de chaînage avant.
2. **Détermination de la fréquence d'arrosage** — calcul automatique et différencié du rythme d'arrosage selon la saison (période de croissance ou période de dormance).
3. **Génération du planning mensuel** — visualisation claire sous forme de calendrier des jours d'arrosage recommandés sur une période de 30 jours.
4. **Alertes et précautions** — détection automatique de situations dangereuses (gel, sur-arrosage, infestation par ravageurs, toxicité, etc.) avec classification par niveau de criticité.

1.3 Choix du domaine d'application

Nous avons choisi de nous spécialiser sur les **plantes grasses et succulentes**, incluant les cactus. Ce choix est motivé par plusieurs facteurs déterminants :

- La richesse et la fiabilité de la base de connaissances disponible auprès de la **Royal Horticultural Society (RHS)** (<https://www.rhs.org.uk/plants>), source de référence en matière d'horticulture.
- La grande variété des comportements d'arrosage observables, allant de « zéro arrosage » pendant la dormance pour certaines espèces à un arrosage hebdomadaire régulier en période de croissance pour d'autres.
- La présence de règles d'alerte nombreuses et pertinentes : sensibilité au gel, toxicité pour les animaux domestiques, ravageurs spécifiques (cochenilles, pucerons, charançons), etc.
- La popularité croissante de ces plantes auprès du grand public, ce qui rend le système expert potentiellement utile et commercialement viable.

1.4 Démarche de recherche documentaire

Pour construire une base de connaissances solide, nous avons mené une recherche approfondie sur le site de la RHS. Pour chaque plante intégrée, nous avons consulté les fiches techniques officielles et extrait les informations suivantes : nom scientifique, famille botanique, forme caractéristique, hauteur maximale, rusticité (température minimale supportée), besoins en eau différenciés selon les saisons, exposition lumineuse requise, type de drainage nécessaire, toxicité éventuelle, ravageurs fréquents, période et couleur de floraison, et besoins en fertilisation. Ces données ont ensuite été structurées sous forme de dictionnaires Python pour alimenter la base de faits du système expert.

2.

Base de connaissances

Base de connaissances

La base de connaissances constitue le cœur du système expert. Ce chapitre détaille sa composition : d'une part la base de faits qui recense les caractéristiques des différentes plantes, d'autre part la base de règles de production qui encode le raisonnement expert permettant d'identifier la plante et de déterminer ses besoins.

2.1 Base de faits

La base de faits contient les descripteurs exhaustifs de **cinq plantes grasses et succulentes** représentatives de la diversité du domaine. Chaque plante est encodée sous forme de dictionnaire Python avec 18 attributs précis. Les sources utilisées sont exclusivement les fiches officielles de la Royal Horticultural Society (RHS), consultées en ligne.

2.1.1 Plantes référencées dans la base

Le tableau ci-dessous présente de manière synthétique les cinq espèces intégrées à la base de faits. Leur sélection a été guidée par leur popularité, leur représentativité morphologique et la diversité de leurs besoins en arrosage.

2.1.2 Structure détaillée d'un fait — Exemple de l'Aloe Vera

Après avoir présenté le tableau récapitulatif des cinq espèces, il est utile de détailler la structure complète d'un fait. Chaque fiche plante est organisée comme un dictionnaire Python structuré. Voici un extrait des attributs les plus importants pour l'inférence :

Attributs du dictionnaire PLANTES["aloe_vera"]	
forme	"rosette"
feuilles_type	"charnues"
feuilles_bordure	"epineuse"
hauteur_max_cm	100
rusticite_min_c	5
arrosage_croissance	"modere"
arrosage_dormance	"tres_faible"
exposition	"plein_soleil"
drainage	"excellent"
toxique	True
tolere_negligence	False
ravageurs	["Cochenilles"]
floraison	"Printemps-Automne"
fertilisation	"mensuelle_mai_aout"

2.2 Base de règles de production

La base de règles constitue la deuxième composante essentielle de la base de connaissances. Elle contient **25 règles de production** réparties en cinq catégories fonctionnelles. Chaque règle est encodée selon le formalisme classique des systèmes experts :

SI $\langle$ condition logique $\rangle$ ALORS $\langle$ action concluante $\rangle$

Les conditions sont implémentées comme des fonctions lambda Python qui reçoivent soit les caractéristiques observées par l'utilisateur, soit les attributs de la plante déjà identifiée. Les actions possibles sont au nombre de cinq : `plante_id` (identification de l'espèce), `classe` (classification morphologique), `arrosage_croissance` (détermination de la fréquence en période de croissance), `arrosage_dormance` (détermination de la fréquence en dormance), et `alerte` (déclenchement d'un message d'avertissement ou de précaution).

2.2.1 Règles d'identification (R01 à R05)

Les règles d'identification constituent la première étape du raisonnement. Elles permettent de déterminer l'espèce de la plante à partir de ses caractéristiques morphologiques observables.

TABLE 2.1 – Règles d'identification (R01 à R05)

ID	Nom	Condition (SI... ALORS...)
R01	Blue Torch Cactus	SI forme = colonnaire ET feuilles = épinées ALORS plante = pilosocereus
R02	Aloe Vera	SI forme = rosette ET feuilles = charnues ET bordure = épineuse ALORS plante = aloe_vera
R03	Echeveria elegans	SI forme = rosette ET feuilles = charnues ET hauteur < 20 cm ALORS plante = echeveria
R04	Crassula ovata	SI forme = buissonnante ET feuilles = charnues ALORS plante = crassula
R05	Sansevieria	SI forme = érigée ET feuilles = rigides ALORS plante = sansevieria

2.2.2 Règles de classification (R06 à R10)

Une fois la plante identifiée, les règles de classification permettent de lui attribuer une classe descriptive qui facilite la compréhension du diagnostic.

TABLE 2.2 – Règles de classification (R06 à R10)

ID	Nom	Condition (SI... ALORS...)
R06	Cactus colonnaire	SI forme = colonnaire ALORS classe = Cactus colonnaire
R07	Succulente rosette	SI forme = rosette ET feuilles = charnues ALORS classe = Succulente rosette
R08	Succulente robuste	SI forme = érigée ET tolère négligence = vrai ALORS classe = Succulente robuste
R09	Succulente arbus-tive	SI forme = buissonnante ALORS classe = Succulente arbustive
R10	Succulente minia-ture	SI hauteur < 20 cm ALORS classe = Succulente miniature

2.2.3 Règles d’arrosage (R11 à R15)

Les règles d’arrosage déterminent la fréquence d’arrosage adaptée à chaque plante, en distinguant la période de croissance de la période de dormance.

TABLE 2.3 – Règles d’arrosage (R11 à R15)

ID	Nom	Condition (SI... ALORS...)
R11	Arrosage libre	SI arrosage_croissance = libre ALORS fréquence = tous les 6 jours
R12	Arrosage modéré	SI arrosage_croissance = modéré ALORS fréquence = tous les 12 jours
R13	Tolérance négli-gence	SI tolère_négligence = vrai ALORS fréquence = tous les 21 jours
R14	Dormance totale	SI arrosage_dormance = pas_du_tout ALORS fréquence = 0 (arrêt total)
R15	Légèrement humide	SI arrosage_croissance = légèrement_humide ALORS fréquence = tous les 14 jours

2.2.4 Règles d’alerte (R16 à R22)

Les règles d’alerte permettent de détecter les situations dangereuses pour la plante et d’informer l’utilisateur.

TABLE 2.4 – Règles d’alerte (R16 à R22)

ID	Nom	Condition (SI... ALORS...)
R16	Risque de gel	SI température ambiante < rusticité_min ALORS DANGER : rentrer la plante
R17	Surexposition	SI exposition = directe ET plante préfère lumière filtrée ALORS ATTENTION : risque brûlure
R18	Toxicité	SI toxique = vrai ALORS ATTENTION : tenir hors de portée enfants/animaux
R19	Mauvais drainage	SI sol = retient_eau ET drainage requis = excellent ALORS DANGER : risque pourriture
R20	Cochenilles	SI symptôme = taches_blanches ALORS DANGER : infestation par cochenilles
R21	Manque d’eau	SI symptôme = feuilles_flétries ALORS ATTENTION : sous-arrosage probable
R22	Sur-arrosage	SI symptôme = feuilles_molles ALORS DANGER : sur-arrosage, suspendre l’arrosage

2.2.5 Règles de précaution (R23 à R25)

Enfin, les règles de précaution fournissent des informations utiles pour l’entretien et la manipulation sécurisée des plantes.

TABLE 2.5 – Règles de précaution (R23 à R25)

ID	Nom	Condition (SI... ALORS...)
R23	Manipulation épines	SI feuilles = épines ALORS INFO : porter des gants épais
R24	Fertilisation	SI fertilisation ≠ non ALORS INFO : rappel fertilisation (mai-août)
R25	Hivernage	SI rusticité < 10°C ALORS INFO : hivernage à l’intérieur obligatoire

3.

Moteur d'inférence

Moteur d'inférence

Le moteur d'inférence est le mécanisme algorithmique qui utilise la base de connaissances pour produire des conclusions à partir des observations de l'utilisateur. Ce chapitre présente le principe du chaînage avant, l'architecture séquentielle adoptée, ainsi que les fonctions clés implémentées.

3.1 Principe du chaînage avant

Le moteur d'inférence retenu pour ce projet est le **chaînage avant** (*forward chaining*), également appelé raisonnement dirigé par les données (*data-driven reasoning*). Ce mode de raisonnement procède de manière incrémentale :

1. Il part des **faits observés** par l'utilisateur (caractéristiques visuelles de la plante, conditions ambiantes, symptômes éventuels).
2. Il applique systématiquement toutes les règles dont les conditions sont satisfaites par les faits actuellement connus.
3. Chaque application de règle produit de nouveaux faits déduits (identification de l'espèce, classe, fréquence d'arrosage, alertes).
4. Le processus itère jusqu'à ce qu'aucune nouvelle règle ne puisse être appliquée.

3.1.1 Flux de raisonnement séquentiel

Le schéma ci-dessous illustre le flux de raisonnement organisé en quatre grandes étapes, chacune correspondant à un sous-ensemble spécifique de règles :

Chaînage avant — Quatre étapes de raisonnement

Entrées observées (caractéristiques visuelles, température, saison, exposition, sol, symptômes)

↓ *Application des règles R01 à R05 : Identification*

Fait déduit N°1 : identifiant unique de la plante (`plante_id`)

↓ *Application des règles R06 à R10 : Classification*

Fait déduit N°2 : classe morphologique descriptive

↓ *Application des règles R11 à R15 : Calcul de l'arrosage*

Fait déduit N°3 : fréquences d'arrosage (croissance et dormance) + planning sur 30 jours

↓ *Application des règles R16 à R25 : Alertes et précautions*

Fait déduit N°4 : liste des alertes classées (DANGER / ATTENTION / INFO)

3.2 Implémentation des quatre étapes

L'architecture du système est organisée en quatre étapes principales permettant de transformer les informations fournies par l'utilisateur en une décision finale. Chaque étape repose sur un ensemble de fonctions et de règles qui assurent le traitement progressif des données et l'exécution du raisonnement.

3.2.1 Étape 1 — Identification de la plante

La fonction `identifier_plante()` parcourt séquentiellement les règles R01 à R05. Pour chaque règle d'identification, elle évalue la condition (implémentée comme une fonction lambda) avec les caractéristiques observées par l'utilisateur : forme générale, type de feuilles, présence d'épines sur les bords, hauteur estimée. Dès qu'une condition est satisfaite, la fonction retourne immédiatement l'identifiant de la plante correspondant et l'identifiant de la règle déclenchée.

```

1 def identifier_plante(forme, feuilles, bordure, hauteur):
2     kwargs = dict(forme=forme, feuilles=feuilles,
3                   bordure=bordure, hauteur=hauteur)
4     for regle in REGLES:
5         if regle["action"] == "plante_id":
6             try:
7                 if regle["condition"](**kwargs):
8                     return regle["valeur"], regle["id"]
9             except Exception:
10                continue
11    return None, "--"

```

Listing 3.1 – Moteur d'identification (chaînage avant — étape 1)

3.2.2 Étape 2 — Classification morphologique

Une fois la plante identifiée, la fonction `determiner_classe()` applique les règles R06 à R10 sur les attributs de la plante (forme, type de feuilles, hauteur, tolérance à la négligence). Cette étape produit une classe descriptive telle que « Succulente miniature », « Cactus colonnaire » ou « Succulente arbustive », qui facilite la compréhension du diagnostic par l'utilisateur.

3.2.3 Étape 3 — Calcul des fréquences d'arrosage

La fonction `calculer_arrosage()` détermine deux valeurs distinctes :

- la **fréquence d'arrosage en période de croissance** (printemps/été), obtenue par application des règles R11, R12, R13 ou R15 selon le type de plante ;
- la **fréquence d'arrosage en période de dormance** (automne/hiver), obtenue par application de la règle R14 (généralement un arrêt total ou un espacement très long).

À partir de la fréquence ainsi déterminée, la fonction génère une liste des jours d'arrosage sur une période de 30 jours par simple itération arithmétique.


```
1 freq = freq_cr if saison == "croissance" else freq_do
2 if freq == 0:
3     return 0, [], "Aucun arrosage en dormance"
4 jours = []
5 j = freq
6 while j <= 30:
7     jours.append(j)
8     j += freq
```

Listing 3.2 – Calcul du planning d’arrosage sur 30 jours

3.2.4 Étape 4 — Évaluation des alertes et précautions

La fonction `evaluer_alertes()` itère sur l’ensemble des règles R16 à R25 et collecte toutes les alertes dont la condition est satisfaite par l’état actuel. Contrairement aux étapes précédentes qui s’arrêtent à la première règle déclenchée, cette étape applique le principe d’**accumulation** : toutes les alertes pertinentes sont conservées et présentées à l’utilisateur.

4.

Interface utilisateur et résultats

Interface utilisateur et résultats

Ce chapitre décrit l'organisation de l'interface graphique et présente des exemples concrets de son utilisation.

4.1 Architecture de l'interface graphique

L'interface est développée avec la bibliothèque Tkinter, incluse nativement dans Python. Elle s'organise en deux panneaux principaux :

- **Panneau gauche** : zone de saisie des entrées utilisateur.
- **Panneau droit** : zone d'affichage des résultats.

4.1.1 Panneau de saisie — Entrées utilisateur

Le panneau gauche permet à l'utilisateur de renseigner l'ensemble des informations nécessaires au diagnostic :

- **Caractéristiques morphologiques** : forme de la plante, type de feuilles, bordure des feuilles, hauteur estimée.

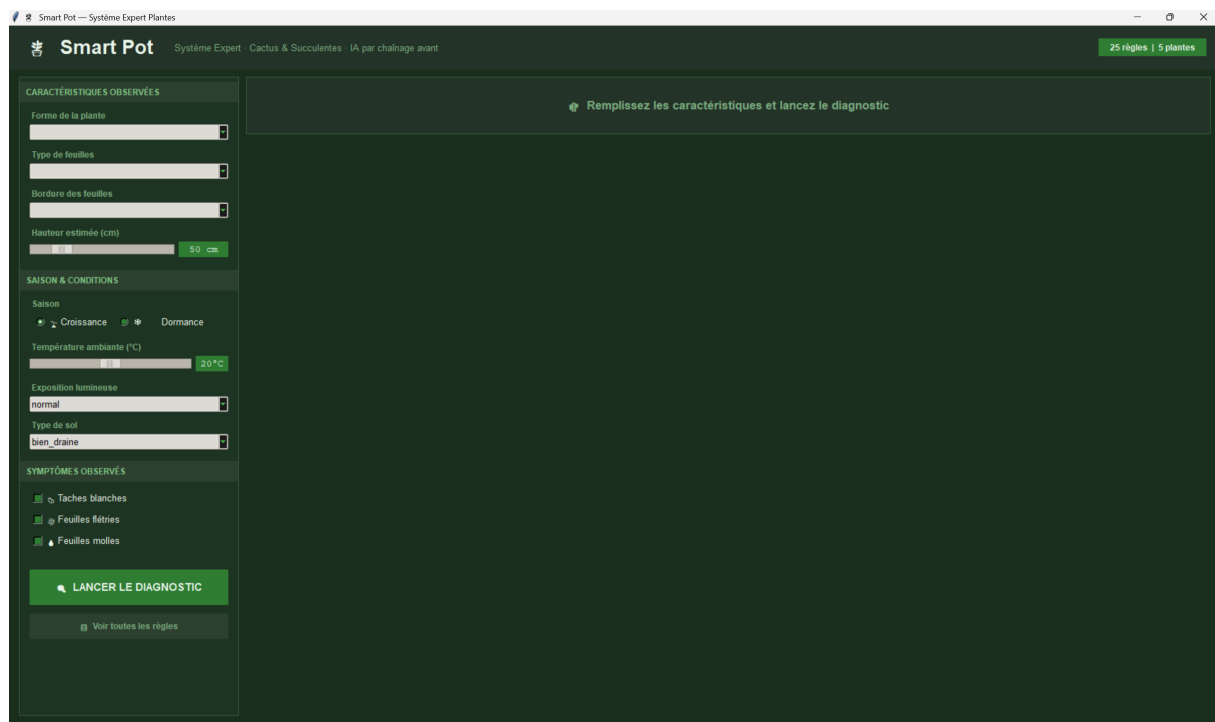


FIGURE 4.1 – Interface graphique principale de l'application Smart Pot

- **Conditions ambiantes** : saison (croissance ou dormance), température ambiante (curseur de -5°C à 45°C), exposition lumineuse (normale, directe, ombre), type de sol (bien drainé, retient l'eau, sableux).
- **Symptômes observés** : cases à cocher pour taches blanches (cochenilles), feuilles flétries (manque d'eau), feuilles molles (sur-arrosage).

4.2 Déroulement du diagnostic

Après avoir renseigné l'ensemble des caractéristiques, l'utilisateur clique sur le bouton « **LANCER LE DIAGNOSTIC** ». Le système expert exécute alors le chaînage avant selon les quatre étapes décrites précédemment, et affiche les résultats dans le panneau droit.

4.2.1 Affichage des résultats

Les résultats du diagnostic sont organisés en trois sections :

- **Fiche plante** : nom commun, nom scientifique, famille, classe attribuée, règle d'identification utilisée, ainsi que l'ensemble des caractéristiques détaillées de la plante.
- **Planning d'arrosage** : fréquences pour les périodes de croissance et de dormance, nombre d'arrosages par mois, et calendrier visuel sur 30 jours avec indication des jours d'arrosage.
- **Alertes et précautions** : liste des alertes déclenchées, classées par niveau DANGER, ATTENTION ou INFO, avec pour chacune un message explicatif et la règle correspondante.

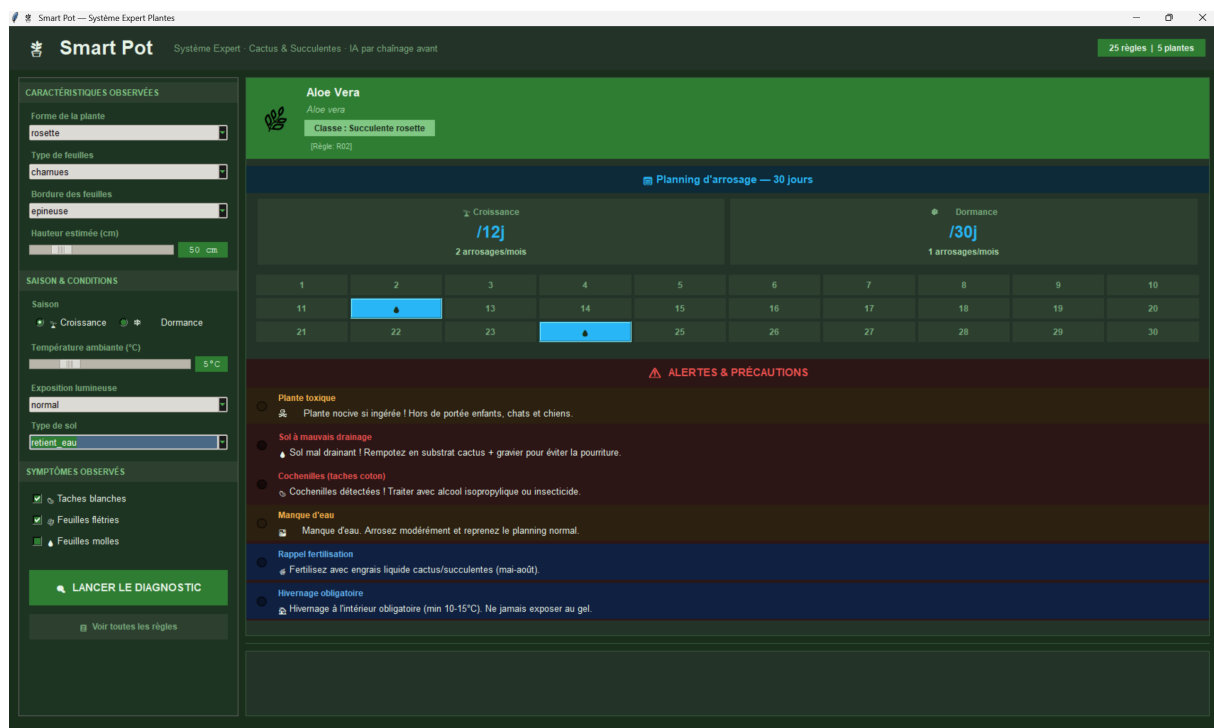


FIGURE 4.2 – Affichage des résultats du diagnostic

Conclusion

Ce mini-projet de Master 1 a abouti à la réalisation complète d'un système expert fonctionnel dédié à la gestion intelligente des plantes grasses et succulentes, répondant ainsi à l'intégralité des objectifs fixés dans le cahier des charges. La base de connaissances comprend 5 espèces documentées via les fiches de la Royal Horticultural Society et 25 règles de production réparties en cinq catégories (identification, classification, arrosage, alertes, précautions). Le moteur d'inférence implémente un chaînage avant organisé en quatre étapes séquentielles, avec une résolution de conflits par priorité d'ordre pour l'identification et par accumulation pour les alertes. L'interface graphique développée avec Tkinter offre une saisie intuitive des caractéristiques et une visualisation claire des résultats sous forme de fiche plante, planning mensuel et alertes classées par criticité. Ce travail constitue une base solide pour une future intégration matérielle (capteurs, actionneur d'arrosage) et une extension à un plus grand nombre d'espèces.

Références documentaires

1. Royal Horticultural Society (RHS). *Plant Finder — Succulents & Cacti*. <https://www.rhs.org.uk/plants>
2. Russell, S. & Norvig, P. (2021). *Artificial Intelligence : A Modern Approach* (4e éd.). Pearson.
3. Jackson, P. (1999). *Introduction to Expert Systems* (3e éd.). Addison-Wesley.
4. Lutz, M. (2013). *Learning Python* (5e éd.). O'Reilly Media.
5. Shipman, J. (2013). *Tkinter 8.5 Reference : a GUI for Python*. New Mexico Tech.